

System Scanner Pro

AI-Powered Distributed Endpoint Monitoring & Remote Scanning Platform

Technical Documentation Package | v3.1

Testing & Maintenance Guide

Automated suite inventory, manual test protocol, and upkeep procedures

Project	System Scanner Pro (v3.1)
Document	Testing & Maintenance Guide
Package date	August 21, 2026
Prepared by	Project Team - Kailasanathan (Team Leader), Saagr, Mega
Audience	Developers, administrators, testers and new team members
Status	Final

This document is part of a nine-volume documentation package generated from the actual project source code. All commands, file paths, endpoints and behaviours described here were verified against the repository at documentation time.

Table of Contents

1. Testing Philosophy & Current State	3
2. Test Environment Setup	3
3. Running the Automated Suite	3
4. Automated Test Inventory	3
5. What the Suite Proves (and What It Does Not)	4
6. Manual Test Programme Overview	4
7. Execution Protocol for a Manual Case	4
8. Results Reporting Rules	4
9. Module 1 - Authentication (Owner: Kailasanathan)	4
10. Module 2 - Dashboard (Owner: Kailasanathan)	4
11. Module 3 - Client Management & Scanning (Owner: Kailasanathan)	5
12. Module 5 - Asset Management (Owner: Saagr)	6
13. Module 7 - License Management (Owner: Saagr)	6
14. Module 9 - Compliance Tracking (Owner: Saagr)	6
15. Module 10 - Alert Management (Owner: Saagr)	6
16. Module 11 - Intelligence & Reports (Owner: Mega)	7
17. Module 12 - Notifications & Audit (Owner: Mega)	7
18. Module 13 - Client Agent (Owner: Kailasanathan)	7
19. Module 14 - WebSocket & Real-Time (Owner: Kailasanathan)	7
20. Module 15 - Health Scoring & Monitoring (Owner: Mega)	7
21. Module 16 - API Testing (Owner: Kailasanathan)	8
22. Module 17 - Integration (Full Team)	8
23. Regression Checklist Before Any Release	9
24. Maintenance Routines Calendar	9
25. Backup Procedures	9
26. Restore Procedure	9
27. Update & Upgrade Procedure	9
28. Monitoring the Monitor (Self-Observability)	9
29. Incident Response Basics	10

1. Testing Philosophy & Current State

The project uses two complementary layers. A Django unit suite locks down the riskiest business rules (registration/approval semantics, cookie fallback, database persistence and auth redirects). A manual program (MANUAL_TESTING.txt/xlsx - 30 essential cases distilled from 100) covers end-to-end behaviour across 17 modules. At documentation time every manual case is marked '[] Pending' in the source workbook; accordingly this volume reports all manual results as [TO BE TESTED]. No result may be claimed without execution evidence.

2. Test Environment Setup

1. Local stack per volume 03 (SQLite mode is fine for tests).
2. python admin/manage.py test scanner_api - the suite creates a temporary database.
3. For agent-involving manual cases, one spare physical machine or VM plus the server host.
4. Optional second browser profile to test concurrent dashboard sessions.

3. Running the Automated Suite

```
cd admin
python manage.py test scanner_api -v 2
```

Expected: all tests pass ('OK') on a clean checkout. Failures indicate environment issues (e.g. legacy SQLite migration path permissions) before they indicate code regressions - read the tracebacks carefully.

4. Automated Test Inventory

test_registration_approval.py - RegistrationApprovalTests (12 tests)

- same_device_re_registration_does_not_keep_old_approval_when_auto_approve_is_off
- existing_key_same_device_demoted_when_never_admin_approved
- existing_key_same_device_keeps_approval_when_admin_approved
- existing_client_moves_to_company_from_new_user_connect_url
- ping_demotes_stale_auto_approval_when_auto_approve_is_off
- pending_client_ping_keeps_pending_status_and_refreshes_last_seen
- explicit_approval_survives_client_ping
- existing_key_different_device_resets_approval
- new_client_is_pending_when_auto_approve_is_off
- blank_hostname_falls_back_to_machine_name
- default_admin_seed_creates_company_and_dynamic_url
- approving_deleted_client_reactivates_and_shows_it

Other modules

File	Tests
test_cookie_auth_fallback.py	authenticates_user_from_signed_cookie; restores_user_after_authentication_middleware
test_persistent_database.py	legacy_database_is_copied_to_persistent_data_dir
test_auth_redirects.py	prefixes_internal_next_paths; prefixes_settings_route_for_company_urls; falls_back_to_prefixed_dashboard_when_next_is_empty; redirects_login_pages_to_prefixed_dashboard; build_url_prefix_path_handles_empty_and_prefixed_values; login_redirect_uses_next_value_without_crashing

Table 1 - Remaining automated tests

5. What the Suite Proves (and What It Does Not)

Proves - approval lifecycle corner cases behave exactly as designed; identity demotion is deterministic; signed-cookie authentication survives middleware ordering; frozen deployments migrate their database.

Does not cover - JWT endpoints, API keys, HMAC agent path, report generation, scheduler jobs, WebSocket consumers, UI behaviour - these rely on the manual programme below.

6. Manual Test Programme Overview

Source of truth: MANUAL_TESTING.txt / MANUAL_TESTING.xlsx (dated 18 July 2026; reduced set of 30 critical cases selected from an original 100). Owners per module are assigned in the workbook: Kailasanathan (auth, dashboard, clients/scanning, agent, websocket, API), Saagr (assets, licenses, compliance, alerts), Mega (reports, notifications/audit, health), Full team (integration). Reproduced below for convenience - always execute from the workbook so statuses stay canonical.

7. Execution Protocol for a Manual Case

1. Prepare environment per section 2; note build/version under test.
2. Execute steps exactly; capture screenshots for pass AND fail evidence.
3. Record Actual Result verbatim (what happened, not interpretation).
4. Set Status: Pass / Fail / Blocked; raise defect ticket for any Fail with repro steps.
5. Update MANUAL_TESTING.xlsx status column - it remains the single source of truth.

8. Results Reporting Rules

Honesty rule

Until a case is executed, its status stays [TO BE TESTED] everywhere (docs, dashboards, reviews). Fabricated 'Pass' entries are treated as process violations.

9. Module 1 - Authentication (Owner: Kailasanathan)

#	Test case	Expected outcome	Status
1.1	Login with valid credentials	Dashboard loads; session established; LoginHistory row written	[TO BE TESTED]
1.2	Login with invalid credentials	Error shown; no session; attempt recorded toward lockout	[TO BE TESTED]
1.3	SQL injection attempts on login fields	Sanitised by ORM; error path; system stable	[TO BE TESTED]
1.4	JWT access + refresh token pair	/api/token/ issues pair; refresh rotates access; expiry enforced	[TO BE TESTED]
1.5	Role-based access (Super/Admin/Viewer)	Permission classes restrict write actions appropriately	[TO BE TESTED]

10. Module 2 - Dashboard (Owner: Kailasanathan)

#	Test case	Expected outcome	Status
2.1	Dashboard loads with stats	Cards/charts render with live counts	[TO BE TESTED]
2.2	Online/offline counts accurate	Counts match device states after heartbeat cycles	[TO BE TESTED]

11. Module 3 - Client Management & Scanning (Owner: Kailasanathan)

#	Test case	Expected outcome	Status
3.1	Client registration & approval	Pending card appears; approval flips online within ~30 s	[TO BE TESTED]
3.2	Auto-approve toggle in settings	Behaviour matches enforced-off implementation (verify UX messaging)	[TO BE TESTED]
3.3	Scan config per client	Interval/enabled persisted; scheduler honours settings	[TO BE TESTED]
3.4	Hardware/software change detection	Diff lists added/removed/changed items between scans	[TO BE TESTED]

12. Module 5 - Asset Management (Owner: Saagr)

#	Test case	Expected outcome	Status
5.1	Create new asset	Asset appears in register as Draft with tag/QR	[TO BE TESTED]
5.2	Status transitions Draft->Pending->Approved->Purchased->Assigned->Retired	Valid moves succeed; invalid jumps rejected with explanation	[TO BE TESTED]
5.3	Assign asset to employee	Assignment recorded; history lineage updated	[TO BE TESTED]

13. Module 7 - License Management (Owner: Saagr)

#	Test case	Expected outcome	Status
7.1	Create license record	Saved with masked key display; seats/expiry stored	[TO BE TESTED]
7.2	License expiration tracking	Expiry surfaces in alerts/dashboard ahead of date	[TO BE TESTED]

14. Module 9 - Compliance Tracking (Owner: Saagr)

#	Test case	Expected outcome	Status
9.1	Compliance checks (ISO/SOC2/GDPR)	Framework views render check/status/evidence	[TO BE TESTED]
9.2	Compliance violation alerts	Violations generate alert entries	[TO BE TESTED]

15. Module 10 - Alert Management (Owner: Saagr)

#	Test case	Expected outcome	Status
10.1	Alert create/edit/delete	CRUD reflected immediately; history preserved	[TO BE TESTED]
10.2	Alert severity levels	Colour/escalation ordering correct	[TO BE TESTED]

16. Module 11 - Intelligence & Reports (Owner: Mega)

#	Test case	Expected outcome	Status
11.1	Key report types (Asset Inventory, Device Health, License Compliance, Audit)	Each generates successfully with sensible content	[TO BE TESTED]
11.2	PDF/Excel/CSV export	All three formats download and open cleanly	[TO BE TESTED]
11.3	Scheduled report create/edit	Schedule persists; generation occurs at cadence	[TO BE TESTED]

17. Module 12 - Notifications & Audit (Owner: Mega)

#	Test case	Expected outcome	Status
12.1	Notifications list loads	Bell/list shows entries; preferences honoured	[TO BE TESTED]
12.2	CRUD actions logged in audit	Actions appear in audit viewer with actor/time	[TO BE TESTED]

18. Module 13 - Client Agent (Owner: Kailasanathan)

#	Test case	Expected outcome	Status
13.1	Agent starts & connects	Registration visible; console logs clean	[TO BE TESTED]
13.2	Auto-discovery via UDP 45000	Server found without URL on same LAN	[TO BE TESTED]
13.3	Heartbeat every 30 seconds	last_seen updates cadence-true	[TO BE TESTED]
13.4	USB/file/process/software monitors	Events stream into device timeline	[TO BE TESTED]
13.5	Offline event queue & replay	Events queued while offline arrive after reconnect	[TO BE TESTED]

19. Module 14 - WebSocket & Real-Time (Owner: Kailasanathan)

#	Test case	Expected outcome	Status
14.1	Agent WS connects (HMAC auth)	Handshake accepted; bad signature rejected	[TO BE TESTED]
14.2	Dashboard WS connects	Live updates flow without refresh	[TO BE TESTED]
14.3	scan_now via WS	Command reaches agent; scan executes	[TO BE TESTED]
14.4	Live status updates	State changes propagate instantly	[TO BE TESTED]

20. Module 15 - Health Scoring & Monitoring (Owner: Mega)

#	Test case	Expected outcome	Status
15.1	Health score calculation (0-100)	Score reflects component inputs; breakdown visible	[TO BE TESTED]
15.2	Failure risk scoring (0-100)	Risk badge consistent with incident factors	[TO BE TESTED]

21. Module 16 - API Testing (Owner: Kailasanathan)

#	Test case	Expected outcome	Status
16.1	API CRUD endpoints work correctly	Create/read/update/delete verified on representative resources	[TO BE TESTED]
16.2	Proper HTTP status codes (200/201/400/401/403/404/500)	Codes match semantics across happy/error paths	[TO BE TESTED]
16.3	API key authentication	sk_ key authorises; inactive/invalid keys rejected	[TO BE TESTED]

22. Module 17 - Integration (Full Team)

Cross-module scenarios defined in the workbook (end-to-end enrollment through report export chains). Execute last, after all module cases pass.

#	Test case	Expected outcome	Status
17.x	As enumerated in MANUAL_TESTING workbook	End-to-end flows succeed	[TO BE TESTED]

23. Regression Checklist Before Any Release

1. python admin/manage.py test scanner_api - green.
2. Boot server locally; login; approve one fresh agent; Scan Now succeeds.
3. Generate one PDF and one Excel report.
4. WebSocket live update observed once (VPS/asgi mode).
5. Backup taken; restore drill on scratch host passes health endpoint check.
6. Version strings bumped consistently (see handover issue M1).

24. Maintenance Routines Calendar

Cadence	Routine	Reference
Daily	Glance executive dashboard; clear criticals; confirm backup success marker	Vol 07 §Executive
Weekly	Pending approvals review; license/warranty horizon scan; disk-full ETA review	Vol 07 §Monitoring
Monthly	Host patching; dependency update review; restore drill; audit-log spot check	This volume §Backup
Quarterly	Credential rotation; anomaly tuning review; prune stale devices/groups	Vol 06 §Policy

Table 16 - Upkeep calendar

25. Backup Procedures

```
# SQLite mode
sudo systemctl stop system-scanner-admin
cp admin/data/scanner.db /backups/scanner-$(date +%F).db
sudo systemctl start system-scanner-admin

# PostgreSQL / Supabase mode
pg_dump "$DATABASE_URL" | gzip > /backups/scanner-$(date +%F).sql.gz
```

Store backups off-host (object storage/NAS). Encrypt archives containing personal or asset data. Keep .env backups separate and encrypted.

26. Restore Procedure

1. Provision target host; install stack per volume 03 (service stopped).
2. SQLite: place .db file at expected path. Postgres: psql restore from dump.
3. Start service; GET /api/health/ returns ok.
4. Spot-check counts vs pre-backup snapshot (clients, scans, assets).

27. Update & Upgrade Procedure

1. Announce window; take backup (section above).
2. git pull; activate venv; pip install -r requirements.txt.
3. python admin/manage.py migrate; systemctl restart system-scanner-admin (or Vercel redeploy).
4. Run regression checklist (section above); rebuild client exe if client/ changed.

28. Monitoring the Monitor (Self-Observability)

- ADMIN self-client row should stay online - if offline, scheduler/UDP listener likely dead.
- Watch local_server.log for repeated exception traces; alert_checker failures appear here.

- Supabase/Vercel dashboards provide platform-side metrics on serverless mode.

29. Incident Response Basics

Incident	First actions	Containment
Suspected credential compromise	Rotate DJANGO_SECRET_KEY + passwords; force logout (sessions invalid)	Review LoginHistory for foreign IPs
Rogue device approved	Reject/delete client; rotate its AgentSecret; investigate events timeline	Check approval audit entries
Data-loss scare	Stop writes; restore latest backup per procedure; diff missing window	Preserve current DB copy for forensics
Serverless abuse (__reset exposure)	Override RESET_TOKEN immediately; review access logs	Consider disabling route until patched

Table 17 - Playbook snippets